High-Level Design Template

Royal Mail Group

Based on the arc42 Template, v9.0

Field	Value
Document Title	
Version	
Status	
Author	
Document Owner	
Classification	Template for Internal Use
Source Requirements	
Template	HLD v0.1

1. Introduction and Goals

Describes the relevant requirements and the driving forces that the software architects and development team must consider:

- Underlying business goals
- Essential features and functional requirements
- Quality goals for the architecture
- Relevant stakeholders and their expectations.

1.1. Requirements Overview

Short description of the functional requirements, driving forces, extract or abstract of requirements. Link to existing requirements documents with version numbers.

Contents

Short description of the functional requirements, driving forces, extract (or abstract) of requirements. Link to (hopefully existing) requirements documents (with version number and information where to find it).

Motivation

From the point of view of the end users a system is created or modified to improve support of a business activity and/or improve the quality.

Form

Short textual description, probably in tabular use-case format. If requirements documents exist this overview should refer to these documents. Keep these excerpts as short as possible. Balance readability of this document with potential redundancy w.r.t to requirements documents.

1.2. Quality Goals

Contents

The top three (max five) quality goals for the architecture whose fulfillment is of highest importance to the major stakeholders. We really mean quality goals for the architecture. Don't confuse them with project goals. They are not necessarily identical.

Motivation

You should know the quality goals of your most important stakeholders, since they will influence fundamental architectural decisions. Make sure to be very concrete about these qualities, avoid buzzwords. If you as an architect do not know how the quality of your work will be judged…

Form

A table with quality goals and concrete scenarios, ordered by priorities

1.3. Stakeholders

Content

Explicit overview of stakeholders - all persons, roles or organizations that

- should know the architecture
- have to be convinced of the architecture
- have to work with the architecture or with code
- need the documentation of the architecture for their work
- have to come up with decisions about the system or its development

Motivation

You should know all parties involved in development of the system or affected by the system. Otherwise, you may get nasty surprises later in the development process. These stakeholders determine the extent and the level of detail of your work and its results.

Form

Table with role names, person names, and their expectations with respect to the architecture and its documentation.

Further Information

See Introduction and Goals *in the arc42 documentation.*

Role/Name	Contact	Expectations
<Role-1>	*<Contact-1>*	*<Expectation-1>*
<Role-2>	*<Contact-2>*	*<Expectation-2>*

2. Architecture Constraints

Contents

Any requirement that constraints software architects in their freedom of design and implementation decisions or decision about the development process. These constraints sometimes go beyond individual systems and are valid for whole organizations and companies.

Motivation

Architects should know exactly where they are free in their design decisions and where they must adhere to constraints. Constraints must always be dealt with; they may be negotiable, though.

Form

Simple tables of constraints with explanations. If needed you can subdivide them into technical constraints, organizational and political constraints and conventions (e.g. programming or versioning guidelines, documentation or naming conventions)

Further Information

See Architecture Constraints *in the arc42 documentation.*

2.1. Technical Constraints

2.2. Organisational & Political Constraints

2.3. Conventions

3. Context and Scope

Contents

Context and scope - as the name suggests - delimits your system (i.e. your scope) from all its communication partners (neighboring systems and users, i.e. the context of your system). It thereby specifies the external interfaces.

If necessary, differentiate the business context (domain specific inputs and outputs) from the technical context (channels, protocols, hardware).

Motivation

The domain interfaces and technical interfaces to communication partners are among your system's most critical aspects. Make sure that you completely understand them.

Form

Various options:

- Context diagrams
- Lists of communication partners and their interfaces.

3.1. Business Context

Contents

*Specification of **all** communication partners (users, IT-systems, ...) with explanations of domain specific inputs and outputs or interfaces. Optionally you can add domain specific formats or communication protocols.*

We need to put context diagram from C4 model but more oriented around interaction between business capability provide by the current system and other organizational business capabilities

Motivation

All stakeholders should understand which data are exchanged with the environment of the system.

Form

All kinds of diagrams that show the system as a black box and specify the domain interfaces to communication partners.

Alternatively (or additionally) you can use a table. The title of the table is the name of your system, the three columns contain the name of the communication partner, the inputs, and the outputs.

<Diagram or Table>

<optionally: Explanation of external domain interfaces>

3.1.1. Business Context - Diagram

A C4-style System Context diagram should be inserted here in production usage.

3.2. Technical Context

Contents

Technical interfaces (channels and transmission media) linking your system to its environment. In addition a mapping of domain specific input/output to the channels, i.e. an explanation which I/O uses which channel.

We need to put context diagram from C4 model but more oriented around interaction between the current system and other RMG systems and external systems.

We must ALSO include **Big Picture** here

Motivation

Many stakeholders make architectural decision based on the technical interfaces between the system and its context. Especially infrastructure or hardware designers decide these technical interfaces.

Form

E.g. UML deployment diagram describing channels to neighboring systems, together with a mapping table showing the relationships between channels and input/output.

<Diagram or Table>

<Optionally: Explanation of technical interfaces>

<Mapping Input/Output to Channels>

Conceptual diagram is replaced with business and technical context diagrams. We will use Context diagrams from C4 model here.

Further Information

See Context and Scope *in the arc42 documentation.*

3.2.1. Technical Context - Diagram

4. Solution Strategy

Contents

A short summary and explanation of the fundamental decisions and solution strategies, that shape system architecture. It includes

- technology decisions
- decisions about the top-level decomposition of the system, e.g. usage of an architectural pattern or design pattern
- decisions on how to achieve key quality goals
- relevant organizational decisions, e.g. selecting a development process or delegating certain tasks to third parties.

Motivation

These decisions form the cornerstones for your architecture. They are the foundation for many other detailed decisions or implementation rules.

Form

Keep the explanations of such key decisions short.

Motivate what was decided and why it was decided that way, based upon problem statement, quality goals and key constraints. Refer to details in the following sections.

Further Information

See Solution Strategy *in the arc42 documentation.*

5. Building Block View

Content

The building block view shows the static decomposition of the system into building blocks (modules, components, subsystems, classes, interfaces, packages, libraries, frameworks, layers, partitions, tiers, functions, macros, operations, data structures, ...) as well as their dependencies (relationships, associations, ...)

This view is mandatory for every architecture documentation. In analogy to a house this is the **Floor Plan**.

Since it contains static view of the system, we need to put container and component diagrams (C2 & C3) from C4 model. It only shows the relationships between various containers, components and external systems (Both internal & external to RMG)

This section should also contain Integration Architecture (Reference to ICDs etc.)

Motivation

Maintain an overview of your source code by making its structure understandable through abstraction.

This allows you to communicate with your stakeholder on an abstract level without disclosing implementation details.

Further Information

See Building Block View *in the arc42 documentation.*

5.1. Container Diagram

5.2. Component Diagram (Backend Components)

6. Runtime View

Contents

The runtime view describes concrete behavior and interactions of the system's building blocks in form of scenarios from the following areas:

- important use cases or features: how do building blocks execute them?
- interactions at critical external interfaces: how do building blocks cooperate with users and neighboring systems?
- operation and administration: launch, start-up, stop
- error and exception scenarios

The main criterion for the choice of possible scenarios (sequences, workflows) is their **architectural relevance**. It is **not** important to describe a large number of scenarios. You should rather document a representative selection.

Motivation

You should understand how (instances of) building blocks of your system perform their job and communicate at runtime. You will mainly capture scenarios in your documentation to communicate your architecture to stakeholders that are less willing or able to read and understand the static models (building block view, deployment view).

Form

Following diagrams could be part of this section:

- **System Landscape Diagram (C4 Model)**
- **Activity Diagrams and Sequence Diagrams for some of the key Use Cases**

Further Information

See Runtime View *in the arc42 documentation.*

6.1. Use Case 01

6.1.1. Activity Diagram – Use Case 01

6.1.2. Sequence Diagram – Use Case 01

6.2. Use Case 02

6.2.1. Activity Diagram – Use Case 02

6.2.2. Sequence Diagram – Use Case 02

7. Deployment View

Content

The deployment view describes:

1. technical infrastructure used to execute your system, with infrastructure elements like geographical locations, environments, computers, processors, channels and net topologies as well as other infrastructure elements and
2. mapping of (software) building blocks to that infrastructure elements.

Often systems are executed in different environments, e.g. development environment, test environment, production environment. In such cases you should document all relevant environments.

Especially document a deployment view if your software is executed as distributed system with more than one computer, processor, server or container or when you design and construct your own hardware processors and chips.

From a software perspective it is sufficient to capture only those elements of an infrastructure that are needed to show a deployment of your building blocks. Hardware architects can go beyond that and describe an infrastructure to any level of detail they need to capture.

This section is split into two: **Deployment View (This Section)** and **Cloud Architecture (Section 8)**

Motivation

Software does not run without hardware. This underlying infrastructure can and will influence a system and/or some cross-cutting concepts. Therefore, there is a need to know the infrastructure.

Form

This section and section 8 should contain Deployment and Integration diagrams. We need to provide Azure deployment diagrams for applications deployed in Azure cloud mapping individual components to Azure services and network topologies. It should also highlight Azure regions, Availability Zones for both Prod and DR (Pre-Prod)

Further Information

See Deployment View *in the arc42 documentation.*

7.1. Deployment Diagram

7.2. System Landscape Diagram

7.3. Infrastructure Architecture

7.3.1. Environments

Environment	Purpose	Notes

7.3.2. Application Components to Infrastructure Components Mapping

Mapping of building blocks to infrastructure (production):

Tier	Components	Building blocks hosted

7.3.3. Quality and performance features:

7.3.4. Target Infrastructure Architecture

7.3.4.1. Platforms Summary

7.3.4.2. Load Balancers

7.3.4.3. IT Infrastructure Landscape View

7.3.4.4. Physical Compute

7.3.4.5. Databases

7.3.4.6. High Available Cluster

7.3.4.7. Network Attached Storage (NAS)

7.3.4.8. Network

7.3.4.9. Local Traffic Manager (LTM)

7.3.4.10. Global Traffic Manager (GTM)

7.3.4.11. Firewall Rules (North-South / East-West)

Details to include:

- DNAT / SNAT rules
- Application rules (FQDN-based)
- Network rules (IP-based)
- Inbound vs outbound distinction

Table 1. Firewall Rules

RULE TYPE	FROM	TO	PORT	ACTION
DNAT	Internet	Web Tier	443	Allow
Network	App Subnet	External API	443	Allow

8. Cloud Architecture

Cloud Infrastructure section should contain cloud architecture diagram highlighting network topology, subnet, firewalls and cloud services used by the architecture. It should also provide information around high availability, regions, subscriptions, SKUs, Network Security Groups, NSG Rules, Firewalls, Cloud patterns and Cost optimization

Availability NFR: This needs to be considered when detailing the Regional/AZ placement

NFRID	Requirement Description	Status

8.1. Regions

Capture the Azure regions used for the solution.

Example 1. Azure Regions

Example

Region	Primary	Secondary	Availability Zones		
			AZ1	AZ2	AZ3
North Europe	Yes				
West Europe					
UK South					
Germany West Central	Yes		Yes	Yes	Yes
Sweden Central		Yes	Yes	Yes	Yes

8.1.1. Rationale (latency, regulatory, capacity constraints)

8.2. Availability Zones

Define high availability architecture.

- Services deployed across zones
- Zone pinning requirements (if any)
- Zonal vs zone-redundant SKUs

Example 2. High Availability Architecture

Example:

- Region supports 3 AZs

- Workloads deployed Zone-Redundant (ZRS / AZ-aware)

- Region: FDS Prod and Prod Application are deployed in the Germany West and Sweden Central regions, each supporting 3 AZs.
- App Service plans

8.3. Subscriptions (Naming & Structure)

Aligned to the **subscription factory model** from your HLD artefact

Details to include:

- Subscription names (fully qualified)
- Environment split:
  - Prod
  - Pre-Prod
  - Non-Prod (Dev/Test)
- Platform vs Workload subscriptions
- Naming standard compliance

Env	Subscription Name	Subscription ID	Type

Standards

8.4. SKU'S Used (Per Region)

Define all critical service SKUs per region.

Details to include:

- Compute SKUs (VM sizes / VMSS)
- Storage SKUs (Standard / Premium / ZRS)
- PaaS SKUs (if applicable)

Example 3. SKU Example

Example:

ENVIRONMENT	COMPONENT	REGION	SKU
NPD	ExpressRoute GW	UK South	ErGw3AZ
PPD	Firewall	UK South	Premium

ENVIRONMENT	COMPONENT	REGION	SKU
PRD	VM	UK South	D4s v5

8.5. Network Security Group (NSG) Rules

Already aligned to your HLD approach, where NSGs control **spoke traffic**

Details to capture:

- Source → Destination
- Port / Protocol
- Action (Allow/Deny)
- Priority
- Purpose

Example 4. NSG Rules

Example table:

RULE NAME	SOURCE	DESTINATION	PORT	ACTION	PURPOSE
Allow-App-To-DB	App Subnet	DB Subnet	1433	Allow	SQL access
Allow-App-To-Blob	App Subnet	Storage Subnet	443	Allow	Blob Access

If traffic stays **inside the VNet → NSG rules must be defined**

If traffic leaves the VNet → **NSGs are not the primary control** (handled via hub firewall(s))

TRAFFIC TYPE	NSG REQUIRED	NOTE
Subnet → Subnet (same VNet)	□ Yes	Primary control
VNet → External / Internet	□ No	Controlled by firewall/routing
VNet → Other VNet	□ No	Controlled via peering + firewall

8.6. Network / Subnet Design

Critical for addressing your “network/subnet size”.

Details to include:

- VNET address space

- Subnet segmentation
- CIDR sizing
- Reserved IP strategy
- Growth headroom

Example 5. Network / Subnet

Example:

SUBNET	CIDR	PURPOSE
AzureFirewallSubnet	/26	Firewall
AppSubnet	/24	App tier
DBSubnet	/24	Database

8.7. Patterns / Blueprints / Catalogue of Whitelisted Azure Products/Services

Pattern Consumption Table

The architect MUST confirm coverage across all domains:

DOMAIN	PATTERN IDENTIFIED (Y/N)	NOTES
Networking		
Security		
Compute		
Data		
Integration		

Catalogue

8.8. Cost Optimisation and FinOps Alignment*

- Use of reserved capacity (where applicable)
- Right-sizing of compute resources
- Use of PaaS over IaaS where appropriate
- Separation of environments for cost control
- Costs MUST be derived from the Azure Calculator
- DR costs MUST be included where applicable

Cost Breakdown Table

Example 6. Cost Breakdown

Example:

COMPONENT	SERVICE	SKU	REGION	EST. MONTHLY COST (£)	JUSTIFICATION
Compute	**VMSS**	**D4s v5**	**UK South**		**Supports app tier scaling**
Security	**Azure Firewall**	**Premium**	**UK South**		**Required for central control**
Data	**Azure SQL**	**Business Critical**	**UK South**		

9. Security Architecture

Content

Software security architecture is the foundational design of a system that embeds defenses directly into the code and infrastructure, ensuring resistance against malicious attacks. It is a critical necessity to proactively mitigate vulnerabilities, prevent catastrophic data breaches, and maintain customer trust

A robust software security architecture is essential for several key reasons:

- **Proactive Vulnerability Mitigation**: *By designing security into the software development life cycle (SDLC) from the ground up, you can prevent up to 80% of security problems before the code is even written. It replaces reactive, "afterthought" patching with built-in resilience.*
- **Minimizing Breach Impact**: *A solid architecture ensures that if a component is breached, the damage is isolated. Principles like "least privilege" and strict access controls prevent an attacker from escalating their access across the entire system.*
- **Protecting Critical Assets**: *It establishes standardized encryption, authentication, and authorization checkpoints to safeguard sensitive user data, intellectual property, and financial systems.*
- **Ensuring Compliance**: *For highly regulated industries like finance, healthcare, and e-commerce, a structured architectural framework is required to meet strict legal regulations and data protection laws*

It should contain the following:

1. Cyber Architecture ARB Survey
2. Data Flow Diagram with STRIDE Threat Model and Mitigation
3. Identity & Access Management (IAM)
4. Data Categorization
5. Usage of AI (If Applicable)
6. Security RAID Log
7. Application Business Criticality Rating
8. Quantum Cryptography Secure (TBC)
9. Active Directory Tier Level (TBC)
10. Privileged Access Requirements (CyberArk)
11. User Provisioning Requirement (SailPoint)
12. Key Management Services Requirements

Motivation

The primary rationale for software security architecture is to proactively embed protective measures into a system, rather than treating them as an afterthought. It acts as a foundational blueprint that

minimizes the attack surface, prevents costly architectural reworks, and ensures business-wide compliance.

The fundamental drivers and objectives of this architecture include:

1. **The Secure by Design Imperative**
    a. **Shift-Left Approach**: *Embedding security during the initial planning and design phases is exponentially more cost-effective than remediating vulnerabilities found in production.*
    b. **Complexity Management**: *Modern software relies heavily on distributed systems and APIs. Security architecture provides consistent, unified controls rather than fragmented, bolt-on solutions that leave systems fragile.*
2. **Core Protection Objectives (CIA Triad)**
    a. **Confidentiality**: *Ensures that sensitive data is protected from unauthorized access, leveraging end-to-end encryption and robust identity access management (IAM).*
    b. **Integrity**: *Maintains the accuracy and trustworthiness of data by implementing strict authorization protocols and data classification policies.*
    c. **Availability**: *Ensures services and data remain accessible, even amidst malicious disruptions or unintended software failures*
3. **Structural Capabilities**
    a. **Segregation & Segmentation**: *Hybrid and cloud environments require independent building blocks so that a compromise in one service does not cascade through the entire organization.*
    b. **Zero-Trust Frameworks**: *Assumes no user or service is trusted by default. Architecture mandates continuous validation, authentication, and authorization.*
4. **Business Enablement and Risk Management**
    a. **Compliance**: *Aligns system designs with data privacy and cybersecurity laws, such as GDPR and other industry frameworks.*
    b. **Cost Reduction**: *Minimizes long-term expenses by preventing security breaches, reducing expensive downtime, and streamlining incident response.*

Form

Security Architecture can be documented through diagrams, text and tables. This section should contain at least a Level-1 DFD along with a STRIDE Threat Model with clearly defined threat boundaries. RAID Logs should be in tabular format.

9.1. Cyber Architecture Standards and NFRs

Table 2. Details

Title	Response
All Royal Mail Cyber Architecture Standards can be found here: Architecture Standards Please confirm that you have reviewed these standards before completing the survey.	Yes/No
TDA Submission Name	
TDA Approval Date Required	
Cyber Advisor Name (A Cyber Advisor is required for every project going through TDA)	
Does this TDA submission introduce any AI? Please detail the requirement.	
Data Privacy Impact Assessment Reference Number (if required)	

Table 3. Cyber Architecture Standards Attestation

Survey Question	Response (Yes/No/NA)	Rationale (If No or NA)
TDA Q1 - Does the solution make use of cloud-based services and ensure that they are appropriately secured and managed inline with ISS-001 - Cloud Computing Security Standard?		
TDA Q2 - Does the solution transfer and store data, if so have you established and documented a minimum acceptable level of cryptographic algorithms and protocols for transmission and storage of Royal Mail data inline with ISS-002 - Cryptography Standard?		

TDA Q3 - Has the solution been designed with the key objectives of protection of data and disaster recovery/business continuity with the ability to recover systems and recent live data in the event of a partial or total loss of a service aligned to ISS-003 - Data Backup and Restoration Standard?		
TDA Q4 -Does the solution ensure that RM assets are appropriately managed and protected from information security breaches inline with ISS-004 - Asset Management Standard?(An Information Asset is a definable piece of information, stored or processed in any manner which is recognized as valuable to the Royal Mail Group. A system that processes that information is also an asset.)		
TDA Q5 - Does the solution required and meet the classifying of information assets as stated in ISS-005 - Information Classification Standard?		
TDA Q6 - Has the solution/design assigned Data/System Impact Levels as stated in ISS-005 - Information Classification Standard?		
TDA Q7 - Does the solution/design require and ensure that data is labelled accordingly as stated in ISS-005 - Information Classification Standard?		
TDA Q 8 - Has the solution or design identified, assessed and provided a treatment plan and monitoring of any Tech and Cyber risk as detailed in ISS-007 - Risk Management Standard?		

TDA Q9 - Has the solution implemented the identified requirements for the provisioning, secure usage, and management of company-provided mobile devices, specifically tablets and smartphones, through the use of Mobile Device Management (MDM) software aligned to ISS-008 - Mobile Security Standard?		
TDA Q10 - Has the solution met the requirements to protect RMG data specifically confidential or strictly confidential data accessed or stored on mobile devices as outlined in ISS-008 - Mobile Security Standard?		
TDA Q11 - Does the solution implement computer network solutions in accordance with the "Information Security requirements" specified in ISS-009 - Network Security Standard?		
TDA Q12 - Has the project understood the procedures necessary to identify, assess, prioritise, and remediate vulnerabilities affecting Royal Mail Group (RMG) following the ISS-010 - Vulnerability Management Standard?(Including but not limited to operating systems, applications, cloud resources, web technologies, server, desktop, mobile device software, hardware, network devices and common coding vulnerabilities in software-development processes)		
TDA Q13 - Has the solution/project reviewed and aligned to the security requirements for developing and using web services, whether in designing a solution, implementing a system or operating a system based on web services in line with ISS-011 - Web Services Security Standard		

TDA Q14 - Does the solution or design implement endpoint solutions in accordance with requirements from Information Security and others; to describe system functionality and parameters necessary to fulfil security requirements; and to define key support processes as specified in ISS-012 - Endpoint Security Standard?		
TDA Q15 - Does the solution or design meet the minimum standard to allow Royal Mail Group (RMG) to address Authentication related risks through the implementation of controls as specified in ISS-013 - Authentication Standard?		
TDA Q16 - Does the solution need to authenticate non-RMG users and is this inline with ISS-013 - Authentication Standard?		
TDA Q17 - Does the solution need to authenticate users from an RMG-managed user directory and does this follow the requirements in ISS-013 - Authentication Standard?		
TDA Q18 - Does the proposed design/solution meet the security requirements to ensure that software development by/on behalf of Royal Mail Group (RMG) minimizes and prevents the risk/impact of information security following ISS-014 - Secure Product Development Standard		
TDA Q19 - Does the design/solution follow the defined set of requirements for security testing providing confirmation that all RMG systems and processes are sufficiently resilient in line with RMG risk appetite, with all controls proven effective as outlined in ISS-015 - Cyber Security Testing Standard?		

TDA Q20 - Have all individuals working on this design/project for Royal Mail Group Ltd, including employees, workers, self-employed contractors, consultants, and agency workers (referred to as “our people”) completed the required cybersecurity training and awareness defined in ISS-016 - Cyber Security Training and Awareness Standard?		
TDA Q21 - Does the design/project follow the secure practices to manage secrets and cryptographic keys following ISS-017 - Secret & Key Management Standard alongside ISS-002 - Cryptography Standard which defines the acceptable cryptographic algorithms and protocols for the secure transmission and storage of data?		
TDA Q22 - Has the design/project determined what software needs to be installed and minimum logging level for Syslog, Common Event Format (CEF) logs and Windows events to maintain visibility across the Royal Mail’s infrastructure. Application logs are based on use case and will be assessed by the Cyber Engineering team using requirements/tools available. i:e Data connector/Logic App etc in line with ISS-020 - SIEM System Monitoring Standard?		
TDA Q23 - Does this design/project meet the requirements for Royal Mail Group (RMG) Cyber Threat Intelligence (CTI) programme and have these been aligned to ISS-023 - Threat Intelligence Standard?		

Cyber Architecture Standards-Aligned NFRs

LeanIX Link to be provided for cyber security NFRs

LeanIX should be used as a reference for the cyber NFRs that need to be met for the solution. The solution should meet all the requirements listed in LeanIX. Any deviations from the mandatory requirements must be documented with a formal

risk acceptance, including compensating controls and an expiry date.

9.2. Data Flow Diagram / Threat Modelling

9.3. STRIDE Mitigation

This section should contain a **STRIDE** Threat Model with clearly defined threat boundaries and mitigation. The **STRIDE** model is a widely used framework for identifying and categorizing potential security threats in software systems. It helps architects and developers systematically analyze the system's architecture to identify vulnerabilities and design appropriate countermeasures. Cyber architects should use the **STRIDE** model to assess the security posture of the system and ensure that it meets the required security standards.They need to work with Solution architects to ensure that the **STRIDE** model is comprehensive and covers all relevant aspects of the system's architecture. The **STRIDE** model should be used in conjunction with other security best practices and standards to ensure that the system is secure and resilient against potential threats. Solution architects must ensure that the **STRIDE** model is integrated into the overall architecture and design of the system, and that it is regularly reviewed and updated as the system evolves. The **STRIDE** model should be used to inform security testing and validation activities, and to ensure that any identified vulnerabilities are addressed in a timely manner.

Table 4. STRIDE Mitigation

STRIDE	Issue	Mitigation	Evidence
Spoofing	**Spoofing: User Identity** - Attacker pretends to be a legitimate OPG/COM user to access FDS. **Spoofing: Service Identity** - Attacker pretends to be a trusted upstream system (MQ/FTPS/JoinedUp).	**Web Application Firewall (WAF)** - Protects FDS frontend and APIs from common web attacks **APIM Security Policies** - Rate limiting, JWT validation, IP filtering, schema validation **Role-Based Access Control (RBAC)** - Ensures least-privilege access for users and services **MQ Authentication & ACLs** - Prevents spoofing and tampering on MQ channels, Service identity validation	

STRIDE	Issue	Mitigation	Evidence
Tampering	**Tampering: API Payloads** - Modification of duty, timesheet or attendance data in transit. **Tampering: Import Files** - Modification of files sent via FTPS before ingestion.	**TLS Encryption** - Encrypts API traffic **APIM Security Policies** - Rate limiting, JWT validation, IP filtering, schema validation, Schema validation **FTPS Hardening** - Ensures secure file transfer with MFA, IP allowlists, and checksums, Checksums & secure transfer **Input Validation** - Prevents SQL injection and malformed payloads, Rejects malformed data	
Repudiation	**Repudiation: User Actions** - Users deny having approved duties or timesheets. **Repudiation: Admin Changes** - Admins deny configuration or security-relevant changes.	**Audit Logging** - Captures user and admin actions for non-repudiation, Immutable logs, Admin activity logging' **SIEM Monitoring** - Detects anomalies, DoS attempts, tampering, and suspicious activity, Alerting on suspicious actions	
Information Disclosure	**Information Disclosure: PII & HR Data** - Exposure of personal and HR data from FDS DB and staging. **Information Disclosure: Integration Data** - Exposure of data in transit via MQ/FTPS/Data Platform.	**TLS Encryption** - Encrypts data in transit **Encryption at Rest** - Protects DB contents, Protects FDS DB, Staging DB, and exported files **Azure Key Vault** - Protects secrets, connection strings, certificates, Protects credentials **Role-Based Access Control (RBAC)** - Ensures least-privilege access for users and services, Restricts DB access	
Denial of Service (DoS)	**DoS: Frontend & APIs** - Overwhelming FDS frontend and APIs with traffic. **DoS: Integration Channels** - Flooding MQ/FTPS to disrupt data flows.	**Web Application Firewall (WAF)** - Protects FDS frontend and APIs from common web attacks, Blocks volumetric attacks **APIM Security Policies** - Rate limiting **SIEM Monitoring** - Detects MQ flooding	

STRIDE	Issue	Mitigation	Evidence
Elevation of Privilege	**Elevation of Privilege: Application** - Exploiting application flaws to gain higher privileges. **Elevation of Privilege: Database** - Abusing DB permissions to bypass application controls.	**Role-Based Access Control (RBAC)** - Ensures least-privilege access for users and services, Restricts backend privileges **Input Validation** - Prevents injection **Azure Key Vault** - Protects DB credentials **Encryption at Rest** - Protects DB files	

9.4. Identity & Access Management (IAM)

This section should explain proposed authentication and authorization mechanism. It should include Role Based Access Control (RBAC), Multi-factor Authentication (MFA) etc. with details around 3rd Party/Admin/Break Glass/Customer/User -User Access Matrix, Groups, Roles, Permissions, Access method, Secure Access Method, MFA etc.

A break glass account (or emergency access account) is a highly privileged IT account designed exclusively for emergencies. It acts as a digital failsafe to bypass standard security controls (like SSO, MFA, or Conditional Access policies) so administrators can restore system access if regular channels fail.

9.5. Usage of AI (If Applicable)

This should contain Purpose of AI (Use Case), Approved use of AI, Securing the AI etc.

9.6. Security RAID Log

ID	Type	Risk / Debt	Level	Mitigation

9.7. Application Business Criticality Rating

9.8. Quantum Cryptography Secure (TBC)

9.9. Active Directory Tier Level (TBC)

9.10. Privileged Access Requirements (CyberArk)

9.11. User Provisioning Requirement (SailPoint)

9.12. Key Management Services Requirements

10. Data Architecture

Content

Data architecture is the blueprint that dictates how an organization acquires, stores, transforms, distributes, and consumes data. It provides a standardized framework that connects business objectives to technical infrastructure, ensuring data is secure, consistent, scalable, and readily accessible for operations and AI applications.

Core Components

A robust data architecture consists of several interconnected building blocks:

- **Data Models**: *Blueprints that map data structures, relationships, and business rules.*
- **Data Flow & Pipelines**: *Systems that ingest, process, and move data from sources to destinations.*
- **Data Storage**: *The repositories where data lives, such as Data Lakes, Data Warehouses, or Lakehouses.*
- **Data Governance & Security**: *Policies, standards, and access controls that maintain data quality and ensure regulatory compliance.*

Motivation

A robust data architecture aligns data systems with business goals, providing a blueprint for data management, storage, and usage. Its core benefits include unified governance, improved data quality, and scalable infrastructure that accelerates real-time analytics and artificial intelligence (AI) adoption.

Key Benefits of Data Architecture

- **Eliminates Data Silos**: *Unifies disparate systems, allowing departments to securely share data and establish a reliable "single version of the truth".*
- **Enhanced Data Quality**: *Enforces consistent data standards and validation rules to minimize duplication and prevent inaccurate reporting.*
- **Scalability & Cost Optimization**: *Optimizes cloud storage and data processing, reducing hardware needs and allocating operational resources efficiently.*
- **Unified Governance & Compliance**: *Implements centralized protocols to secure sensitive information and meet regulatory frameworks like GDPR.*
- **Accelerated Innovation**: *Provides a reliable, controlled foundation for advanced technologies like AI, machine learning, and real-time predictive analytics.*

Form

*Data Architecture should be captured in form of **Data Models** and **Data Flow Diagrams** alongside a bunch of information captured in various tables and text paragraphs. The following sections outline the key information that should be captured for any new interface or data product created.*

10.1. Data Architecture Blueprint

10.1.1. Conceptual Data Models

High-level abstract representations showing core business entities and their relationships.

10.1.2. Data Flow Diagram

Visual maps illustrating where data originates, the paths it takes through pipelines, and final consumption points.

10.1.3. Storage Strategies

Locations and technologies for data at rest, categorized by workload (e.g., Relational, NoSQL, Data Platform, Caching).

10.2. Integration and Processing

10.2.1. Integration Patterns

Defined approaches for data movement (e.g., ETL vs. ELT, batch processing, real-time streaming).

10.2.2. Data Sources and Destinations

Explicit listings of all external/internal input sources and target destinations.

10.2.3. Ingestion Methods

Tools and scenarios used to acquire the data (e.g., full loads, incremental syncing, API pulls).

For any new interface

10.3. Ownership model (data owner, data steward, data consumer)

Table 5. Data Ownership Model

Interface	Owner	Steward	Consumer

10.4. Policies: Data classification (public, internal, confidential, restricted), Retention (e.g., 5 years), Compliance (GDPR, HIPAA, etc.)

Table 6. Data Policies

Data Classification	
Data Categorization	
Retention Policy	
Data Compliance	

Specify any new data product created or any of the existing data products used. For each data product:

10.5. Purpose / business use case

10.6. Product owner (business + technical)

10.7. Consumers (teams, applications)

10.8. Value statement (why it exists)

10.9. Mapping between new interface and data product

Data Product Architecture

10.10. Data product boundaries (e.g., domain-specific, cross-domain)

10.11. Interaction between data products

10.12. Event-driven / batch flows

11. Cross-cutting Concepts

Content

This section describes crosscutting concepts (practices, patterns, regulations or solution ideas). Such concepts are often related to multiple building blocks.

Motivation

*Concepts form the basis for **conceptual integrity** (consistency, homogeneity) of the architecture. Thus, they are an important contribution to achieve inner qualities of your system.*

This is the place in the template that we provided for a cohesive specification of such concepts.

Many of these concepts relate to or influence several of your building blocks.

Form

The form can be varied:

- concept papers with any kind of structure
- example implementations,especially for technical concepts
- cross-cutting model excerpts or scenarios using notations of the architecture views

Structure

*Pick **only** the most-needed topics for your system and assign each a level-2 heading in this section (e.g. 8.1, 8.2 etc).*

11.1. <*Concept 1*>

<explanation>

11.2. <*Concept 2*>

<explanation>

...

11.3. <*Concept n*>

<explanation>

DO NOT ATTEMPT to cover all of the topics of the aforementioned diagram.

Further Information

Some topics within systems often concern multiple building blocks, hardware elements or

*development processes. It might be easier to communicate or document such **cross-cutting** topics at a central location, instead of repeating them in the description of the concerned building blocks, hardware elements or development processes. Certain concepts might concern **all** elements of a system, others might only be relevant for a few. In the diagram above, logging concerns all three components, whereas security is relevant only for two components.*

See Concepts *in the arc42 documentation.*

Example 7. Cross-Cutting Concepts

- Identity, Authentication and Authorisation
- Audit and Traceability
- Integration Resilience
- Data Privacy and Retention
- Logging, Monitoring and Observability
- User-experience consistency across the merged web app shell

12. Architecture Decisions

Contents

Important, expensive, large scale or risky architecture decisions including rationales. With "decisions" we mean selecting one alternative based on given criteria.

Please use your judgement to decide whether an architectural decision should be documented here in this central section or whether you better document it locally (e.g. within the white box template of one building block).

Avoid redundancy. Refer to section 4, where you already captured the most important decisions of your architecture.

Only capture those decisions that deviates from well established patterns and practices. We also need to capture decisions where better option was available but we had to stick with approved pattern.

Motivation

Stakeholders of your system should be able to comprehend and retrace your decisions.

Form

Various options:

- ADR Documenting Architecture Decisions for every important decision
- List or table, ordered by importance and consequences or:
- more detailed in form of separate sections per decision

12.1. ADR-001 - Title of the decision

Table 7. ADR-001

Aspect	Detail
Status	Accepted/Rejected/Proposed
Context	
Decision	
Consequences	

12.2. ADR-002 - Title of the decision

Table 8. ADR-002

Aspect	Detail
Status	Accepted/Rejected/Proposed

Aspect	Detail
Context	
Decision	
Consequences	

13. Quality Requirements

Content

This section contains all relevant quality requirements.

*The most important of these requirements have already been described in section 1.2. (quality goals), therefore they should only be referenced here. In this section 10 you should also capture quality requirements with lesser importance, which will not create high risks when they are not fully achieved (but might be **nice-to-have**).*

Motivation

Since quality requirements will have a lot of influence on architectural decisions you should know what qualities are really important for your stakeholders, in a specific and measurable way.

13.1. Quality Requirements Overview

Content

An overview or summary of quality requirements.

This section should provide an overview of quality requirements at a category level (Like performance, scalability, interoperability, security etc.) and how those requirements are met by the proposed architecture

Motivation

Often we encounter dozens (or even hundreds) of detailed quality requirements. In this overview section you should try to summarize, e.g. by describing categories or topics (as suggested by ISO 25010:2023 or Q42)

If these summary descriptions are already precise, specific enough and measurable, you may skip section 10.2.

Form

*Use a simple table in which each line contains a category or topic and a short description of the quality requirement. Alternatively, you may use a mind map to structure these quality requirements. In literature, the idea of a **quality attribute tree** has also been described, which puts the generic term "quality" as the root and uses a tree-like refinement of the term "quality". [Bass+21] introduced the term **"Quality Attribute Utility Tree"** for this purpose.*

Category (ISO 25010)	Quality requirement
Functional Suitability	
Performance Efficiency	
Compatibility	
Usability	

Category (ISO 25010)	Quality requirement
Reliability	
Security	
Maintainability	
Portability	

13.2. Quality Scenarios (selected)

Content

Quality scenarios make quality requirements concrete and allow to decide whether they are fulfilled (in the sense of acceptance criteria). Ensure that your scenarios are specific and measurable.

This section is complementary to NFRs defined in LeanIX. This provides an acceptance criteria for each NFR that could be tested. The distinction between an NFR and Quality Scenarios could be understood with the following examples:

> **NFR** – All PII data should be encrypted at rest
>
> **Quality Scenario** – All PII data should be encrypted using AES256 before storing in the database.
>
> In the above example – Just by looking at the NFR alone, we cannot test it but the associated quality scenario makes it testable
>
> **NFR** – All save operations should be complete in less than 5 seconds
>
> **Quality Scenario** – Upon clicking on Save button on employee edit screen, record should be saved within 3 seconds and success message should be displayed on the screen.

*Key point here is **"measurable"**. We can link either LeanIX or any other form of document(s) where these quality scenarios are captured.*

Two kinds of scenarios are especially useful:

- **Usage scenarios** *(also called application scenarios or use case scenarios) describe the system's runtime reaction to a certain stimulus. This also includes scenarios that describe the system's efficiency or performance. Example: The system reacts to a user's request within one second.*
- **Change scenarios** *describe the desired effect of a modification or extension of the system or of its immediate environment. Example: Additional functionality is implemented or requirements for a quality attribute change, and the effort or duration of the change is measured.*

Form

Typical information for detailed scenarios include the following:

In short form (favoured in the Q42 model):

- **Context/Background**: *What kind of system or component, what is the envirionment or situation?*
- **Source/Stimulus**: *Who or what initiates or triggers a **behaviour**, **reaction** or **action**.*
- **Metric/Acceptance Criteria**: *A response including a **measure** or **metric**.*

The long form of scenarios (favoured by the SEI and [Bass+21]) is more detailed and includes the following information:

- **Scenario ID**: *A unique identifier for the scenario.*
- **Scenario Name**: *A short, descriptive name for the scenario.*
- **Source**: *The entity (user, system, or event) that initiates the scenario.*
- **Stimulus**: *The triggering event or condition the system must address.*
- **Environment**: *The operational context or condition under which the system experiences the stimulus.*
- **Artifact**: *The building-blocks or other elements of the system affected by the stimulus.*
- **Response**: *The outcome or behavior the system exhibits in reaction to the stimulus.*
- **Response Measure**: *The criteria or metric by which the system's response is evaluated.*

All **NFRs** should be linked to the quality scenarios. If a quality scenario is not linked to any NFR, it should be captured in the architecture document. **LeanIX** will contain all quality requirements and quality scenarios. Solution architects should ensure that all quality scenarios are captured in **LeanIX** and linked to the relevant NFRs and provide a link to **LeanIX** in this section. If any quality scenario is not captured in **LeanIX**, it should be captured in this section of the architecture document.

14. RAID Log and Technical Debts

Contents

A list of identified technical risks, issues and technical debts, ordered by priority alongside mitigation plan for each one of them

Arc42 says to log risks here as rest of the things (Assumptions, Issues and Dependencies) are project management tools.

Assumptions and dependencies anyway should be gone before Gate 5 because if these are not resolved by then, they become risks or issues to the project.

We do not want to remove assumptions and dependencies but these must be linked to respective risks and issues

Motivation

> Risk management is project management for grown-ups
>
> — Tim Lister, Atlantic Systems Guild

This should be your motto for systematic detection and evaluation of risks and technical debts in the architecture, which will be needed by management stakeholders (e.g. project managers, product owners) as part of the overall risk analysis and measurement planning.

Form

List of risks and/or technical debts, probably including suggested measures to minimize, mitigate or avoid risks or reduce technical debts.

ID	Type	Risk / Debt	Level	Mitigation

15. Glossary

Contents

The most important domain and technical terms that your stakeholders use when discussing the system. You can also see the glossary as source for translations if you work in multi-language teams.

Motivation

You should clearly define your terms, so that all stakeholders

- have an identical understanding of these terms
- do not use synonyms and homonyms

Form

*A table with columns <**Term**> and <**Definition**>. Potentially more columns in case you need translations.*

Term	Definition